

Map-Reduce to Aggregation Pipeline

An aggregation pipeline provides better performance and usability than a map-reduce operation.

Map-reduce operations can be rewritten using aggregation pipeline stages such as `$group` and `$merge`.

For map-reduce operations that require custom functionality, MongoDB provides the `$accumulator` and `$function` aggregation operators. Use these operators to define custom aggregation expressions in JavaScript.

Map-reduce expressions can be re-written as shown in the following sections.

Map-Reduce to Aggregation Pipeline Translation Table

The table is only an approximate translation. For instance, the table shows an approximate translation of `mapFunction` using the `$project`.

- However, the `mapFunction` logic may require additional stages, such as if the logic includes iteration over an array:

```
function() {
  this.items.forEach(function(item){ emit(item.sku, 1); });
}
```

Then, the aggregation pipeline includes an `$unwind` and a `$project`:

```
{ $unwind: "$items "},
{ $project: { emits: { key: { "$items.sku" }, value: 1 } } } },
```

- The `emits` field in `$project` may be named something else. For visual comparison, the field name `emits` was chosen.

Map-Reduce	Aggregation Pipeline
<pre> db.collection.mapReduce(<mapFunction>, <reduceFunction>, { query: <queryFilter>, sort: <sortOrder>, limit: <number>, finalize: <finalizeFunction>, out: <collection> }) </pre>	<pre> db.collection.aggregate([{ \$match: <queryFilter> }, { \$sort: <sortOrder> }, { \$limit: <number> }, { \$project: { emits: { k: <expression>, v: <expression> } } }, { \$unwind: "\$emits" }, { \$group: { _id: "\$emits.k", value: { \$accumulator: { init: <initCode>, accumulate: <reduceFunction>, accumulateArgs: ["\$emit.v"], merge: <reduceFunction>, </pre>

Map-Reduce

Aggregation Pipeline

```
        finalize: <finalizeFunction>,
        lang: "js" }}
    }},
    { $out: <collection> }
  ])
}
```

```
db.collection.mapReduce(
  <mapFunction>,
  <reduceFunction>,
  {
    query: <queryFilter>,
    sort: <sortOrder>,
    limit: <number>,
    finalize: <finalizeFunction>,
    out: { replace: <collection>, db:<db> }
  }
)
```

```
db.collection.aggregate([
  { $match: <queryFilter> },
  { $sort: <sortOrder> },
  { $limit: <number> },
  { $project: { emits: { k: <expression>, v: <expression> } } },
  { $unwind: "$emits" },
  { $group: {
    _id: "$emits.k",
    value: { $accumulator: {
      init: <initCode>,
      accumulate: <reduceFunction>,
      accumulateArgs: [ "$emit.v" ],
      merge: <reduceFunction>,
      finalize: <finalizeFunction>,
      lang: "js" }}
  } },
  { $out: { db: <db>, coll: <collection> } }
])
```

```
db.collection.mapReduce(
  <mapFunction>,
  <reduceFunction>,
  {
    query: <queryFilter>,
    sort: <sortOrder>,
    limit: <number>,
    finalize: <finalizeFunction>,
    out: { merge: <collection>, db: <db> }
  }
)
```

```
db.collection.aggregate([
  { $match: <queryFilter> },
  { $sort: <sortOrder> },
  { $limit: <number> },
  { $project: { emits: { k: <expression>, v: <expression> } } },
  { $unwind: "$emits" },
  { $group: {
    _id: "$emits.k",
    value: { $accumulator: {
      init: <initCode>,
      accumulate: <reduceFunction>,
      accumulateArgs: [ "$emit.v" ],
      merge: <reduceFunction>,
      finalize: <finalizeFunction>,
      lang: "js" }}
  } },
  { $merge: {
    into: { db: <db>, coll: <collection> },
    on: "_id"
    whenMatched: "replace",
    whenNotMatched: "insert"
  } },
])
```

```
db.collection.mapReduce(
  <mapFunction>,
  <reduceFunction>,
  {
    query: <queryFilter>,

```

```
db.collection.aggregate([
  { $match: <queryFilter> },
  { $sort: <sortOrder> },
  { $limit: <number> },
  { $project: { emits: { k: <expression>, v: <expression> } } },

```

Map-Reduce

```
sort: <sortOrder>,
limit: <number>,
finalize: <finalizeFunction>,
out: { reduce: <collection>, db: <db> }
}
)
```

Aggregation Pipeline

```
{ $unwind: "$emits" },
{ $group: {
  _id: "$emits.k",
  value: { $accumulator: {
    init: <initCode>,
    accumulate: <reduceFunction>,
    accumulateArgs: [ "$emit.v" ],
    merge: <reduceFunction>,
    finalize: <finalizeFunction>,
    lang: "js" }}
}},
{ $merge: {
  into: { db: <db>, coll: <collection> },
  on: "_id"
  whenMatched: [
    { $project: {
      value: { $function: {
        body: <reduceFunction>,
        args: [
          "$_id",
          [ "$value", "$$new.value" ]
        ],
        lang: "js"
      }}
    }}
  ]
  whenNotMatched: "insert"
}},
])
```

```
db.collection.mapReduce(
  <mapFunction>,
  <reduceFunction>,
  {
    query: <queryFilter>,
    sort: <sortOrder>,
    limit: <number>,
    finalize: <finalizeFunction>,
    out: { inline: 1 }
  }
)
```

```
db.collection.aggregate([
  { $match: <queryFilter> },
  { $sort: <sortOrder> },
  { $limit: <number> },
  { $project: { emits: { k: <expression>, v: <expression> } } },
  { $unwind: "$emits" },
  { $group: {
    _id: "$emits.k",
    value: { $accumulator: {
      init: <initCode>,
      accumulate: <reduceFunction>,
      accumulateArgs: [ "$emit.v" ],
      merge: <reduceFunction>,
      finalize: <finalizeFunction>,
      lang: "js" }}
  }}
])
```

Examples

Various map-reduce expressions can be rewritten using aggregation pipeline operators, such as `$group`, `$merge`, and others, without requiring custom functions. However, for illustrative purposes, the following examples provide both alternatives.

Example 1

The following map-reduce operation on the `orders` collection groups by the `cust_id`, and calculates the sum of the `price` for each `cust_id`:

```
var mapFunction1 = function() {
  emit(this.cust_id, this.price);
};
var reduceFunction1 = function(keyCustId, valuesPrices) {
  return Array.sum(valuesPrices);
};
db.orders.mapReduce(
  mapFunction1,
  reduceFunction1,
  { out: "map_reduce_example" }
)
```

Alternative 1: (Recommended) You can rewrite the operation into an aggregation pipeline without translating the map-reduce function to equivalent pipeline stages:

```
db.orders.aggregate([
  { $group: { _id: "$cust_id", value: { $sum: "$price" } } },
  { $out: "agg_alternative_1" }
])
```

Alternative 2: (For illustrative purposes only) The following aggregation pipeline provides a translation of the various map-reduce functions, using `$accumulator` to define custom functions:

```
db.orders.aggregate([
  { $project: { emit: { key: "$cust_id", value: "$price" } } }, //
    equivalent to the map function
  { $group: { // equivalent to the reduce function
    _id: "$emit.key",
    valuesPrices: { $accumulator: {
      init: function() { return 0; },
      initArgs: [],
      accumulate: function(state, value) { return state + value; },
      accumulateArgs: [ "$emit.value" ],
      merge: function(state1, state2) { return state1 + state2; },
      lang: "js"
    } }
  } },
  { $out: "agg_alternative_2" }
])
```

1. First, the `$project` stage outputs documents with an `emit` field. The `emit` field is a document with the fields:
 - `key` that contains the `cust_id` value for the document
 - `value` that contains the `price` value for the document

```
{ "_id" : 1, "emit" : { "key" : "Ant 0. Knee", "value" : 25 } }
{ "_id" : 2, "emit" : { "key" : "Ant 0. Knee", "value" : 70 } }
{ "_id" : 3, "emit" : { "key" : "Busby Bee", "value" : 50 } }
{ "_id" : 4, "emit" : { "key" : "Busby Bee", "value" : 25 } }
{ "_id" : 5, "emit" : { "key" : "Busby Bee", "value" : 50 } }
{ "_id" : 6, "emit" : { "key" : "Cam Elot", "value" : 35 } }
{ "_id" : 7, "emit" : { "key" : "Cam Elot", "value" : 25 } }
{ "_id" : 8, "emit" : { "key" : "Don Quis", "value" : 75 } }
{ "_id" : 9, "emit" : { "key" : "Don Quis", "value" : 55 } }
{ "_id" : 10, "emit" : { "key" : "Don Quis", "value" : 25 } }
```

2. Then, the `$group` uses the `$accumulator` operator to add the emitted values:

```
{ "_id" : "Don Quis", "valuesPrices" : 155 }
{ "_id" : "Cam Elot", "valuesPrices" : 60 }
{ "_id" : "Ant 0. Knee", "valuesPrices" : 95 }
{ "_id" : "Busby Bee", "valuesPrices" : 125 }
```

3. Finally, the `$out` writes the output to the collection `agg_alternative_2`. Alternatively, you could use `$merge` instead of `$out`.

Example 2

The following map-reduce operation on the `orders` collection groups by the `item.sku` field and calculates the number of orders and the total quantity ordered for each sku. The operation then calculates the average quantity per order for each sku value and merges the results into the output collection.

```
var mapFunction2 = function() {
  for (var idx = 0; idx < this.items.length; idx++) {
    var key = this.items[idx].sku;
    var value = { count: 1, qty: this.items[idx].qty };
    emit(key, value);
  }
};

var reduceFunction2 = function(keySKU, countObjVals) {
  reducedVal = { count: 0, qty: 0 };
  for (var idx = 0; idx < countObjVals.length; idx++) {
    reducedVal.count += countObjVals[idx].count;
    reducedVal.qty += countObjVals[idx].qty;
  }
  return reducedVal;
};

var finalizeFunction2 = function (key, reducedVal) {
  reducedVal.avg = reducedVal.qty/reducedVal.count;
  return reducedVal;
};

db.orders.mapReduce(
  mapFunction2,
  reduceFunction2,
  {
    out: { merge: "map_reduce_example2" },
    query: { ord_date: { $gte: new Date("2020-03-01") } },
    finalize: finalizeFunction2
  }
);
```

Alternative 1: (Recommended) You can rewrite the operation into an aggregation pipeline without translating the map-reduce function to equivalent pipeline stages:

```
db.orders.aggregate( [
  { $match: { ord_date: { $gte: new Date("2020-03-01") } } },
  { $unwind: "$items" },
  { $group: { _id: "$items.sku", qty: { $sum: "$items.qty" }, orders_ids:
    { $addToSet: "$_id" } } },
  { $project: { value: { count: { $size: "$orders_ids" }, qty: "$qty",
    avg: { $divide: [ "$qty", { $size: "$orders_ids" } ] } } } },
  { $merge: { into: "agg_alternative_3", on: "_id", whenMatched:
    "replace", whenNotMatched: "insert" } }
] )
```

Alternative 2: (For illustrative purposes only) The following aggregation pipeline provides a translation of the various map-reduce functions, using `$accumulator` to define custom functions:

```
db.orders.aggregate( [
  { $match: { ord_date: { $gte: new Date("2020-03-01") } } },
  { $unwind: "$items" },
  { $project: { emit: { key: "$items.sku", value: { count: { $literal: 1
    }, qty: "$items.qty" } } } },
  { $group: {
    _id: "$emit.key",
    value: { $accumulator: {
      init: function() { return { count: 0, qty: 0 }; },
      initArgs: [],
      accumulate: function(state, value) {
        state.count += value.count;
        state.qty += value.qty;
        return state;
      },
      accumulateArgs: [ "$emit.value" ],
      merge: function(state1, state2) {
        return { count: state1.count + state2.count, qty: state1.qty +
          state2.qty };
      },
      finalize: function(state) {
        state.avg = state.qty / state.count;
        return state;
      },
      lang: "js"
    }
  } },
  { $merge: {
    into: "agg_alternative_4",
    on: "_id",
    whenMatched: "replace",
    whenNotMatched: "insert"
  } }
] )
```

1. The `$match` stage selects only those documents with `ord_date` greater than or equal to `new Date("2020-03-01")`.
2. The `$unwind` stage breaks down the document by the `items` array field to output a document for each array element. For example:

```
{ "_id" : 1, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-01T00:00:00Z"), "price" : 25, "items" : { "sku" : "oranges", "qty" : 5, "price" : 2.5 }, "status" : "A" }
{ "_id" : 1, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-01T00:00:00Z"), "price" : 25, "items" : { "sku" : "apples", "qty" : 5, "price" : 2.5 }, "status" : "A" }
{ "_id" : 2, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 70, "items" : { "sku" : "oranges", "qty" : 8, "price" : 2.5 }, "status" : "A" }
{ "_id" : 2, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 70, "items" : { "sku" : "chocolates", "qty" : 5, "price" : 10 }, "status" : "A" }
{ "_id" : 3, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 50, "items" : { "sku" : "oranges", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 3, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 50, "items" : { "sku" : "pears", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 4, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-18T00:00:00Z"), "price" : 25, "items" : { "sku" : "oranges", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 5, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-19T00:00:00Z"), "price" : 50, "items" : { "sku" : "chocolates", "qty" : 5, "price" : 10 }, "status" : "A" }
...
```

3. The `$project` stage outputs documents with an `emit` field. The `emit` field is a document with the fields:
 - `key` that contains the `items.sku` value
 - `value` that contains a document with the `qty` value and a `count` value

```
{ "_id" : 1, "emit" : { "key" : "oranges", "value" : { "count" : 1, "qty" : 5 } } }
{ "_id" : 1, "emit" : { "key" : "apples", "value" : { "count" : 1, "qty" : 5 } } }
{ "_id" : 2, "emit" : { "key" : "oranges", "value" : { "count" : 1, "qty" : 8 } } }
{ "_id" : 2, "emit" : { "key" : "chocolates", "value" : { "count" : 1, "qty" : 5 } } }
{ "_id" : 3, "emit" : { "key" : "oranges", "value" : { "count" : 1, "qty" : 10 } } }
{ "_id" : 3, "emit" : { "key" : "pears", "value" : { "count" : 1, "qty" : 10 } } }
{ "_id" : 4, "emit" : { "key" : "oranges", "value" : { "count" : 1, "qty" : 10 } } }
{ "_id" : 5, "emit" : { "key" : "chocolates", "value" : { "count" : 1, "qty" : 5 } } }
```

...

4. The `$group` uses the `$accumulator` operator to add the emitted `count` and `qty` and calculate the `avg` field:

```
{ "_id" : "chocolates", "value" : { "count" : 3, "qty" : 15, "avg" : 5 } }
{ "_id" : "oranges", "value" : { "count" : 7, "qty" : 63, "avg" : 9 } }
{ "_id" : "carrots", "value" : { "count" : 2, "qty" : 15, "avg" : 7.5 } }
{ "_id" : "apples", "value" : { "count" : 4, "qty" : 35, "avg" : 8.75 } }
{ "_id" : "pears", "value" : { "count" : 1, "qty" : 10, "avg" : 10 } }
```

5. Finally, the `$merge` writes the output to the collection `agg_alternative_4`. If an existing document has the same key `_id` as the new result, the operation overwrites the existing document. If there is no existing document with the same key, the operation inserts the document.

TIP

See also:

[Aggregation Commands Comparison](#)